

Final Project Report

Group Members:

Boyd, Aidan Sebastian

Burks, Nykaela R

Jindal, Geetanshi

Nguyen, Vivian P

1. Introduction

The project is related to peer-to-peer file sharing protocol based on the similar lines of the well-known bit torrent protocol. The report contains the final progress made by the team towards completion of the project. This document details the contributions of each group member, code design discussion, screenshots of the implementation, and a guide on how to run the program.

2. Group Member Roles and Contributions

The roles and contributions are listed below, sorted alphabetically by last name:

Name (and contribution)	Role	Key Contributions
Boyd, Aidan Sebastian Contribution: 25%	Peer File Download, Multithreading	Builds peer files download logic, make downloading multithread, Handle failed TCP Connections
Burks, Nykaela R Contribution: 25%	Peer: Tracker Communication & Client Thread	Primary architect for peers client server thread, sets messages and handle responses, send requests for the list of files
Jindal, Geetanshi Contribution: 25%	Peer Upload File and Integration	Build peer server side thread, enforce 1024 - byte limit scripts for demo, integration of peer functions, config files
Nguyen, Vivian P Contribution: 25%	Tracker Server Lead	Primary architect for the tracker server, implements logic to receive and respond to

Name (and contribution)	Role	Key Contributions
		commands,creates files to store peer data

3. Citations of the sources of any code, library, package used.

No external library or packages used

4. Code Design

Tracker Server:

Main Architecture:

The tracker server first reads the sconfig to get the port number and shared directory. It uses this information to create a socket to start listening onto this port. Each peer connects to this port # and gets its own pthread where it can communicate with the tracker server via tracker commands.

Tracker File Format:

A tracker file is a plain text file that has the fields Filename, Filesize, Description, MD5, and peer list of who has the file (in format: ip:port:startbyte:endbyte:timestamp)

Peer Program (Client Thread):

Main Architecture:

The peer program first reads clientThreadConfig.cfg to get the tracker server's IP address, port number, and the periodic update interval. Using this information it establishes TCP socket connections to the tracker server on demand — meaning a fresh connection is opened for each command sent and closed immediately after the response is received. The peer program runs a main loop that accepts manual user commands for testing purposes, and also spawns a background pthread that periodically sends updatetracker messages to the tracker server at the interval specified in the config file.

Tracker Commands:

The peer program can send four commands to the tracker server:

- **REQ LIST** is sent to request the full list of tracker files currently available on the tracker server. The peer reads the response until it receives the <REP LIST END> marker and displays the results to the user.
- **GET** is sent to request a specific tracker file from the tracker server. The peer reads the response between the <REP GET BEGIN> and <REP GET END md5> markers, verifies the MD5 checksum of the received content, and saves the tracker file to its local cache for use in peer-to-peer downloading.
- **createtracker** is sent when the peer has a complete file it wants to share. It sends the filename, filesize, description, MD5, IP address, and port number to the tracker server which then creates a .track file entry for that file.
- **updatetracker** is sent periodically by the background update thread to keep the peer's entry fresh on the tracker server. It sends the filename, current start byte, and end byte of the file chunk the peer currently has. This allows the tracker to always have an up to date picture of what each peer is sharing.

Config File Format:

The peer reads from two config files. `clientThreadConfig.cfg` contains three lines — the tracker server port number, the tracker server IP address, and the periodic update interval in seconds. `serverThreadConfig.cfg` contains the port number the peer listens on for incoming peer connections and the name of the peer's shared folder.

Connection Design:

Each command opens a new TCP socket connection to the tracker, sends the command, reads the full response, then closes the socket. This keeps the implementation simple and matches the tracker server's design where each worker thread serves one peer request then terminates. All socket connections use `AF_INET` and `SOCK_STREAM` for reliable TCP communication.

Resume Logic:

On startup the peer checks its local cache for any incomplete downloads from previous sessions. If incomplete tracker files are found it will attempt to resume downloading the remaining bytes rather than starting the entire download over from scratch.

5. Installation guide and running guide

Need to do:

- Need pictures of the clients relating to each other & tracker

Installation requires a Unix based operating system like Mac or Linux, and 1 device. First to get started, you will clone the repository on this link [GitHub_Repository](#) and open a terminal on your OS prompt. Navigate to the directory you have chosen to clone the repository onto. Then input the following command on the terminal, "bash demo_onemachine.sh". There will be 14 terminals that will open up, 1 tracker and 3 peers. No additional libraries or packages are required.

To revert the directory back to its original state before demo_onemachine.sh then run "clean.sh".

If the compilation fails ensure the GCC is up to date, make sure the command is correct, make sure you are in the right directory.

OUTPUT

Peer 1: Upload small file

```
Peer1: Tracker 127.0.0.1:3490 upload_port=8001 shared=shared
Peer1: Checking cache for incomplete downloads...
Peer1: No incomplete downloads

Peer1> Peer1: [Upload] Listening on port 8001
Peer1: createtracker work.txt 909 small_test_file cb1db557b0acdd169d055913bf47b46d 127.0.0.1 8001
Peer1: Tracker: <createtracker succ>

Peer1> Peer1: Tracker: <updatetracker work.txt succ>

Peer1> Peer1: [Upload] Served bytes 0-908 from shared/work.txt
Peer1: Exiting
```

Peer 2: Upload large file

```
Peer2: Tracker 127.0.0.1:3490 upload_port=8002 shared=shared
Peer2: Checking cache for incomplete downloads...
Peer2: No incomplete downloads

Peer2> Peer2: [Upload] Listening on port 8002
Peer2: createtracker demo.mp4 10755034 large_test_file 77aa3ae11ab927ab985edb187de67a11 127.0.0.1 8002
Peer2: Tracker: <createtracker succ>

Peer2> Peer2: Tracker: <updatetracker demo.mp4 succ>

Peer2> Peer2: [Upload] Served bytes 57344-65535 from shared/demo.mp4
Peer2: [Upload] Served bytes 131072-139263 from shared/demo.mp4
```

Peer 3: Download small and large file

```
Peer3: Tracker 127.0.0.1:3490 upload_port=8003 shared=shared
Peer3: Checking cache for incomplete downloads...
Peer3: No incomplete downloads

Peer3> Peer3: [Upload] Listening on port 8003
Peer3: List

--- Available Files ---
<REP LIST 2>
<1 work.txt 909 cb1db557b0acdd169d055913bf47b46d>
<2 demo.mp4 10755034 77aa3ae11ab927ab985edb187de67a11>
<REP LIST END>

Peer3> Peer3: Get work.txt.track
Peer3: Receiving tracker file: work.txt.track
Peer3: MD5 verified OK for work.txt.track
Peer3: Parsing: cache/work.txt.track
Peer3: Downloading work.txt size=909 segments=1
Peer3: [Segment 1/1] bytes 0-908
Peer3: downloading 0 to 908 bytes of work.txt from 127.0.0.1 8001
Peer3: Tracker: <updatetracker work.txt succ>
Peer3: Download complete: shared/work.txt (909 bytes)
Peer3: Deleted cache entry: cache/work.txt.track
```